otto group
one.O

DB Performance Tuning

Firefighting or Fixing Root Causes?

- Peter Ramm

May 2026

About Otto Group one.O

Fusion to one.O
2025

Parent company
Otto Group

Locations
Dresden, Hamburg, Altenkunstadt, Madrid, Málaga, Valencia, Taipei, Hyderabad

Number of employees
around 1,000

Management Board
Katrin Behrens, Dr. Stefan Borsutzky

About me

Peter Ramm

Software architect / Team lead at One.O in Dresden

More than 35 years of experience in IT-projects

Main focus:

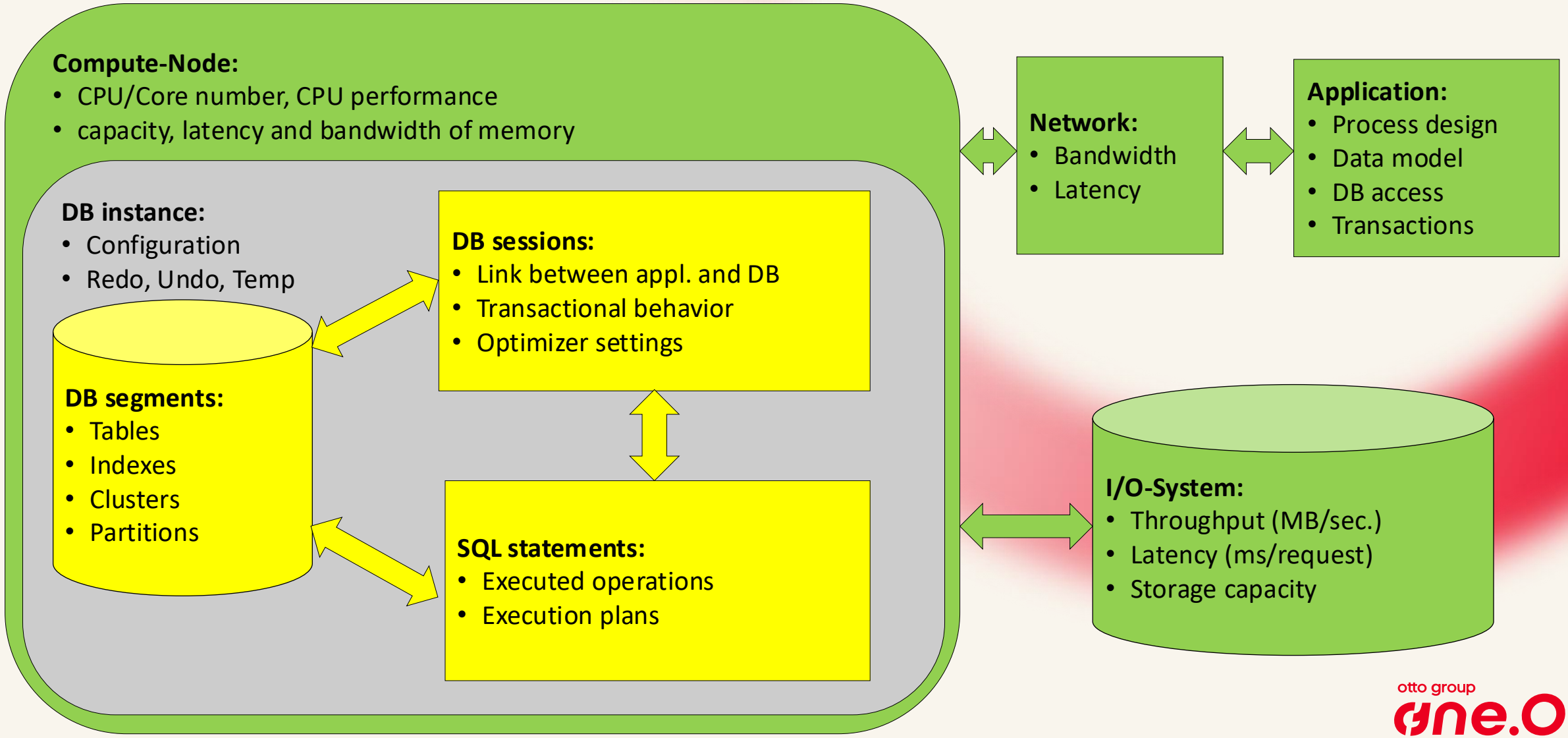
- Development of OLTP systems based on Oracle databases
- From architecture consulting up to trouble shooting
- Performance analysis and optimization of working systems

Oracle Ace Associate since 2024

Mail: Peter.Ramm@og1o.de

otto group one.O

Factors influencing performance in DB use



otto group one.O

Oracle‘s solution for metrics and statistics

- Dynamic performance views: Vxxx, GVxxx
- Show current state, but not the history

- StatsPack: own rudimentary snapshots, available since release 8i
- Automatic Workload Repository (AWR), since release 10g
- Active Session History (ASH), since release 10g

- AWR and ASH are tailor-made for troubleshooting and forensics
- But require licensing of Enterprise Edition + Diagnostics Pack
- Alternative: Panorama-Sampler

Panorama for Oracle Databases

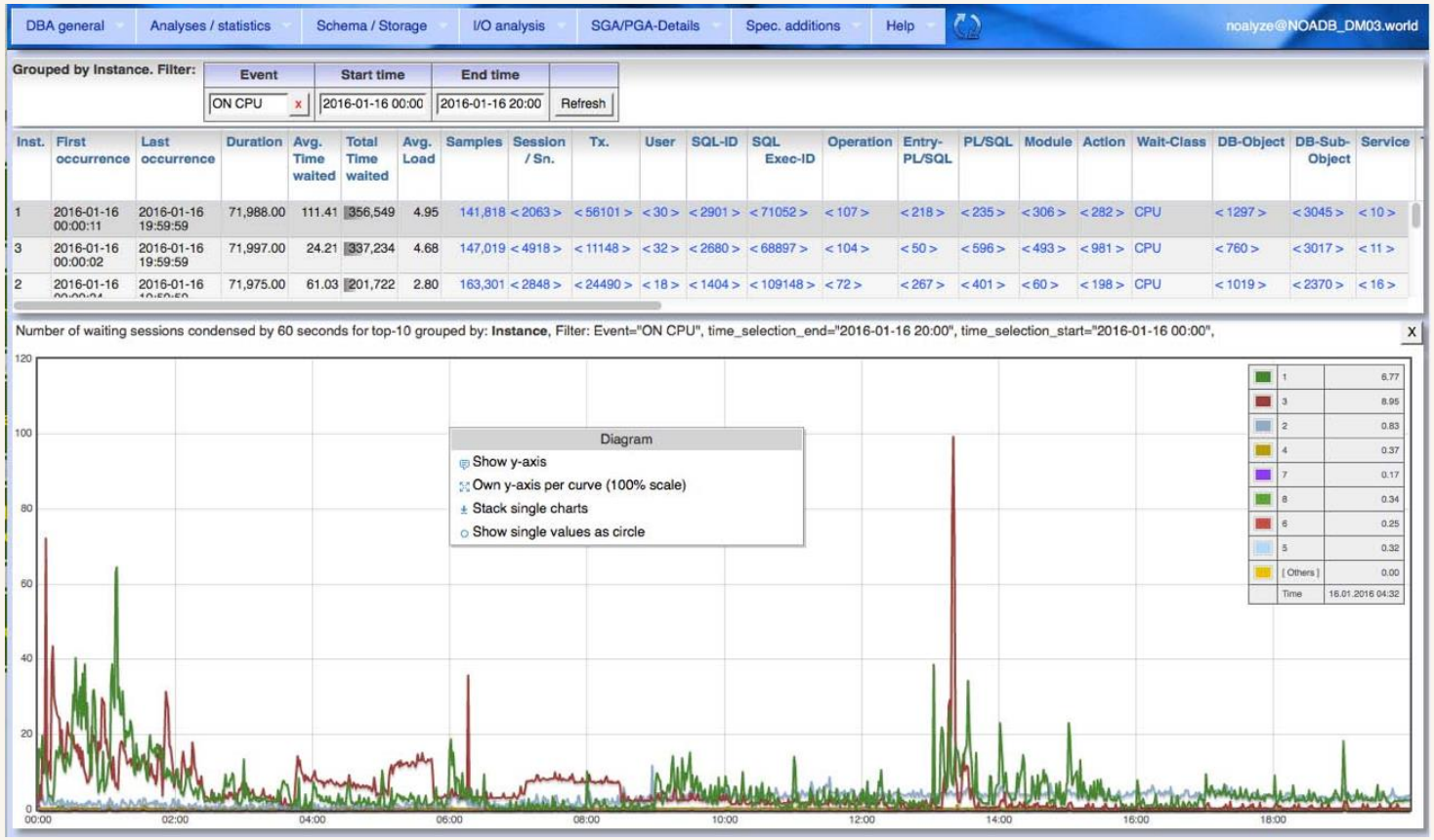
Tool for performance analysis of Oracle DB Free of charge (GPL3)

Based on dynamic performance views and
- AWR recordings
- or Panorama‘s own sampling
(for use with SE or without Diagnostics Pack)

Docker image or self-starting jar file

Panorama accesses your DB on a read-only basis and does not install any DB objects of its own. (except using Panorama-Sampler)

So you can test the functions without any risk.



Description of Panorama incl. download link : https://rammpeter.github.io/panorama.html
Oracle performance analysis blog : https://rammpeter.blogspot.com
Various slides on the use of Panorama : https://www.slideshare.net/PeterRamm1

otto group one.O

Approaches for performance improvement

Reactive (Firefighting)

- Such tools like Panorama also support in incident-related investigation
- e.g. after breaking SLA thresholds or receiving business complaints
- Reaction occurs mostly only after performance problems become visible
- Only the currently visible ocurrences of a problem pattern are fixed

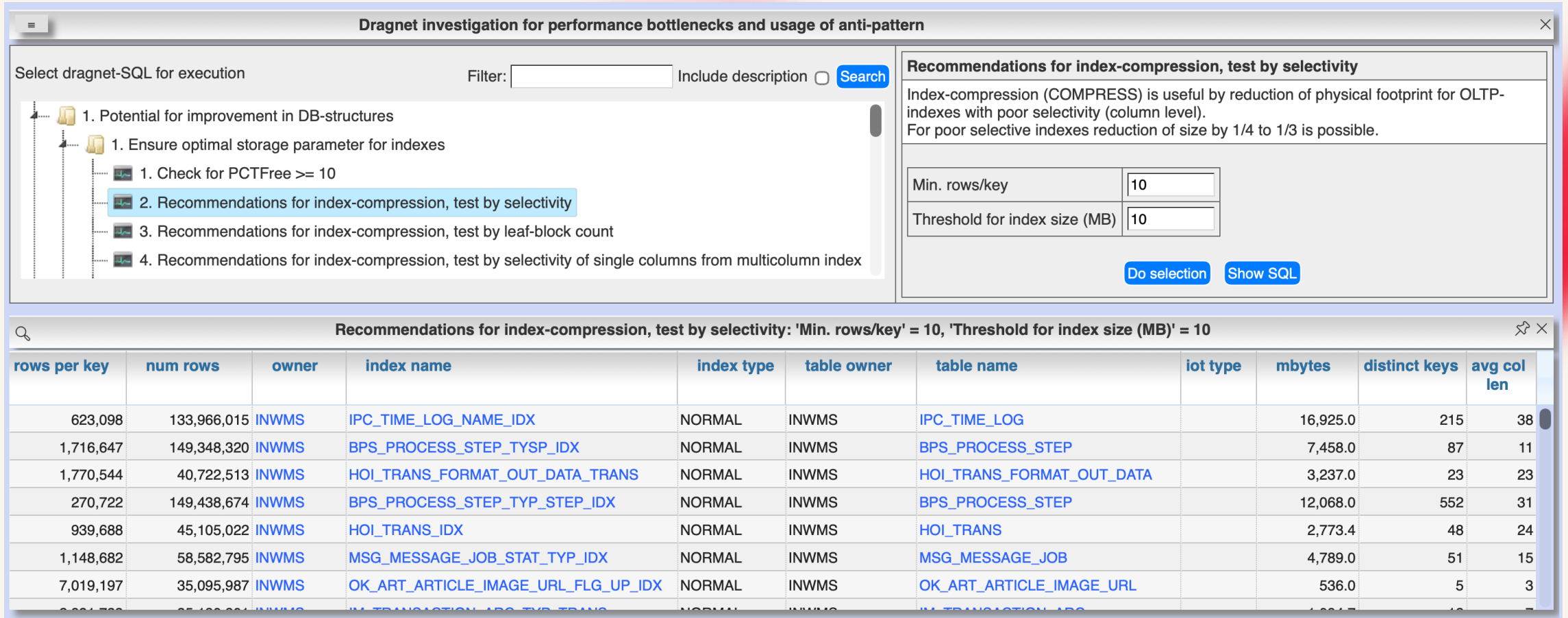
Proactive (Fixing before escalation)

- A lot of performance-related scenarios can be detected by evaluating the traces in the database
- A list of occurrences for a certain scenario ordered by severity gives us a chance to:
  - Rate the relevance of a suboptimal behavior in terms of performance
  - Check which lightweight solution could possibly exist to fix the issue
  - Estimate the potential benefit of a fix
- Considering the same type of problem row by row often allows to apply the same solution approach multiple times

otto group one.O

Examples

- Now let's pick some of the 140+ checks as examples
- They are live demonstrated on a not yet fully optimized production system



rows per key	num rows	owner	index name	index type	table owner	table name	iot type	mbytes	distinct keys	avg col len
623,098	133,966,015	INWMS	IPC_TIME_LOG_NAME_IDX	NORMAL	INWMS	IPC_TIME_LOG		16,925.0	215	38
1,716,647	149,348,320	INWMS	BPS_PROCESS_STEP_TYSP_IDX	NORMAL	INWMS	BPS_PROCESS_STEP		7,458.0	87	11
1,770,544	40,722,513	INWMS	HOI_TRANS_FORMAT_OUT_DATA_TRANS	NORMAL	INWMS	HOI_TRANS_FORMAT_OUT_DATA		3,237.0	23	23
270,722	149,438,674	INWMS	BPS_PROCESS_STEP_TYP_STEP_IDX	NORMAL	INWMS	BPS_PROCESS_STEP		12,068.0	552	31
939,688	45,105,022	INWMS	HOI_TRANS_IDX	NORMAL	INWMS	HOI_TRANS		2,773.4	48	24
1,148,682	58,582,795	INWMS	MSG_MESSAGE_JOB_STAT_TYP_IDX	NORMAL	INWMS	MSG_MESSAGE_JOB		4,789.0	51	15
7,019,197	35,095,987	INWMS	OK_ART_ARTICLE_IMAGE_URL_FLG_UP_IDX	NORMAL	INWMS	OK_ART_ARTICLE_IMAGE_URL		536.0	5	3

View VSQL, VSQLArea or DBA_Hist_SQLStat for various checks

- These metrics per SQL tell us e.g. :
  - The most time-consuming SQLs
  - SQLs with highest elapsed time per single execution
  - SQLs with most needed buffer gets per result row

Resource-intensive SQL statements from gv$SQLArea

Hit limit 100 | Filter | User | SQL-ID | Sorted by Total number of parse calls | PL/SQL | Show SQL

SQL of current SGA from GV$SQLArea, grouped by SQL-ID, instance='1'

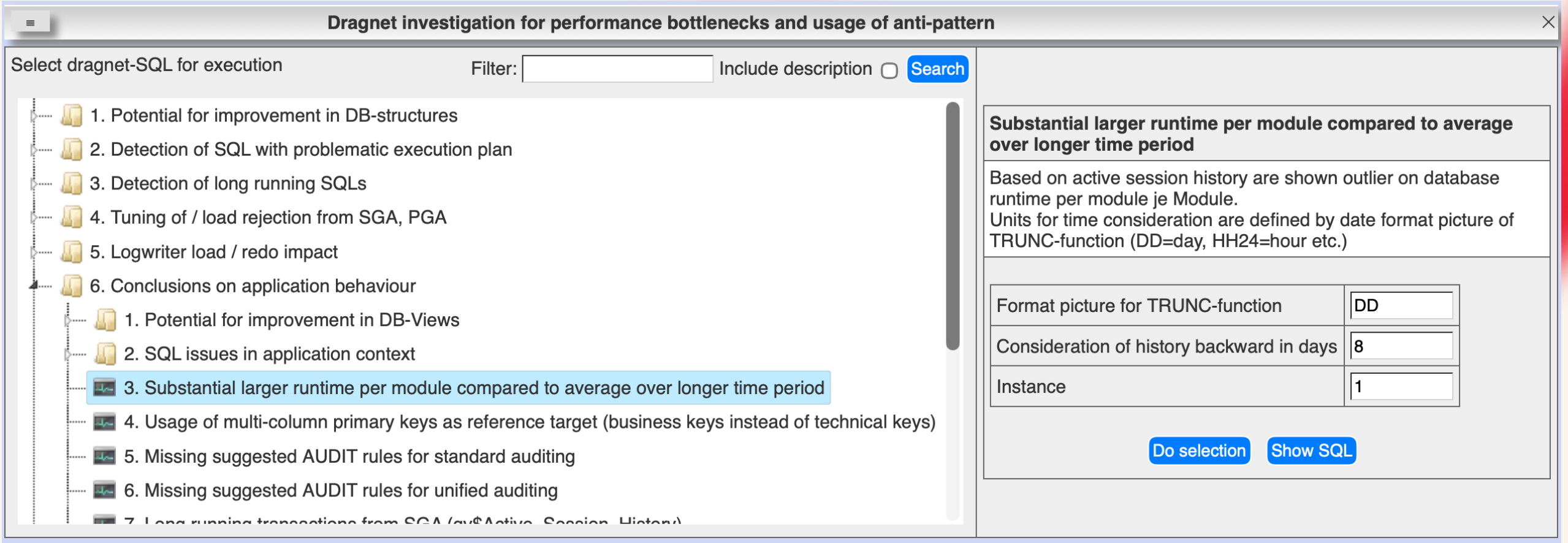
Con-ID	SQL-ID	SQL-Text	C	V	P	Last active	User	Parse	Execs	Elapsed	Ela./Ex.	CPU	Disk Reads	Disk/Ex.	Buffer Gets	Buffer/Ex.	Rows proc.	Rows/Ex.	Parses	Stat
3	ar62gmg36kucd	SELECT u.* FROM ok_art_article_	1	1	1	2026-05-05 11:42:19	INWMS	INWMS	566,025,592	323,086	0.001	28,883	582,501,687	1	1,831,214,825	3	62,380	0	566,025,592	VA
3	0nvdnx5b3rhsf	SELECT ALT_CONNECT_SERVER, ALT_FORMAT_SE	1	1	1	2026-05-05 11:42:19	INWMS	INWMS	233,664,573	7,442	0.000	2,486	2,906	0	467,292,739	2	233,643,375	1	233,659,944	VA
3	2taagj1ypkc5d	SELECT t0.FLG_ACTIVE, t0.ID_CRE, t0.ID_M	1	1	1	2026-05-05 11:42:19	INWMS	INWMS	3,155,967,641	385,732	0.000	164,013	299	0	2,634,954,213	1	3,155,438,337	1	210,494,150	VA
3	1y63d55t1dkch	SELECT BEAN_NAME, ID_CRE, ID_UPD, TS_CRE	1	1	1	2026-05-05 11:42:19	INWMS	INWMS	82,664,236	2,461	0.000	886	2,934	0	165,326,058	2	82,662,676	1	82,663,779	VA
3	57dzb4s4fhyhg	SELECT CL_EXECUTE, ID_CRE, ID_TRANS_TYPE	1	1	1	2026-05-05 11:42:19	INWMS	INWMS	78,612,955	2,952	0.000	750	1,925	0	157,222,086	2	78,610,194	1	78,612,670	VA
3	4xnum2hhx46k6	SELECT CL_LU, FLG_ACTIVE, FLG_LU_FOR_PAC	1	1	1	2026-05-05 11:42:19	INWMS	INWMS	56,877,810	1,910	0.000	707	15	0	113,753,173	2	56,875,518	1	46,048,755	VA
3	fvdrg1r2krbty	SELECT PARA_VALUE FROM TO_PARA_CONFIG WH	1	1	1	2026-05-05 11:42:19	INWMS	INWMS	47,438,969	1,632	0.000	677	254	0	142,316,132	3	47,438,616	1	44,549,526	VA
3	a7xdn51cyr2gg	SELECT FLG_STRAPPING, ID_CRE, ID_UPD, LO	1	1	1	2026-05-05 11:42:19	INWMS	INWMS	40,556,220	1,057	0.000	309	58	0	54,850,773	1	22,238,366	1	39,944,165	VA
3	6s7rnp7gqrfa8	SELECT t1.AISLE_SRC, t1.ARTVAR, t1.BARCO	1	1	1	2026-05-05 11:42:20	INWMS	INWMS	39,808,555	19,122	0.000	15,708	119	0	2,485,807,000	62	883,345	0	39,808,628	VA

Number of rows processed per execute

otto group one.O

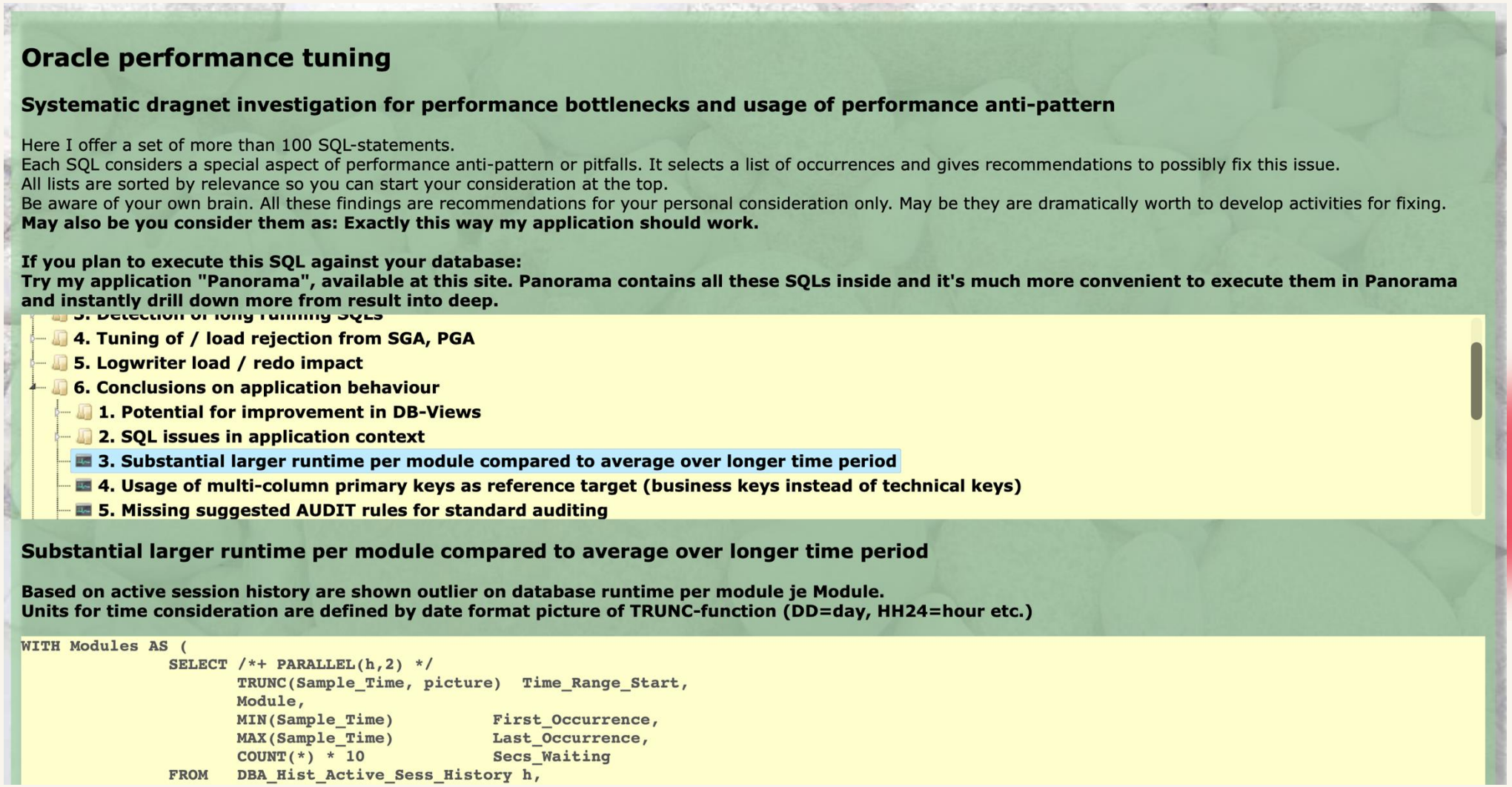
Dragnet investigation for performance antipattern

- Besides the main analysis features (SQL, object statistics, ASH etc.), Panorama also has a dedicated block that checks for individual antipatterns in DB use.
- 140+ predefined checks for occurrences of a certain problem scenario



Dragnet investigation: Standalone SQLs without Panorama

- The SQLs used for dragnet investigation are also available outside Panorama
  https://rammpeter.github.io/oracle_performance_tuning.html



Oracle performance tuning

Systematic dragnet investigation for performance bottlenecks and usage of performance anti-pattern

Here I offer a set of more than 100 SQL-statements.
Each SQL considers a special aspect of performance anti-pattern or pitfalls. It selects a list of occurrences and gives recommendations to possibly fix this issue.
All lists are sorted by relevance so you can start your consideration at the top.
Be aware of your own brain. All these findings are recommendations for your personal consideration only. May be they are dramatically worth to develop activities for fixing.
May also be you consider them as: Exactly this way my application should work.

If you plan to execute this SQL against your database:
Try my application "Panorama", available at this site. Panorama contains all these SQLs inside and it's much more convenient to execute them in Panorama and instantly drill down more from result into deep.

- 4. Tuning of / load rejection from SGA, PGA
- 5. Logwriter load / redo impact
- 6. Conclusions on application behaviour
  - 1. Potential for improvement in DB-Views
  - 2. SQL issues in application context
  - 3. Substantial larger runtime per module compared to average over longer time period
  - 4. Usage of multi-column primary keys as reference target (business keys instead of technical keys)
  - 5. Missing suggested AUDIT rules for standard auditing

Substantial larger runtime per module compared to average over longer time period

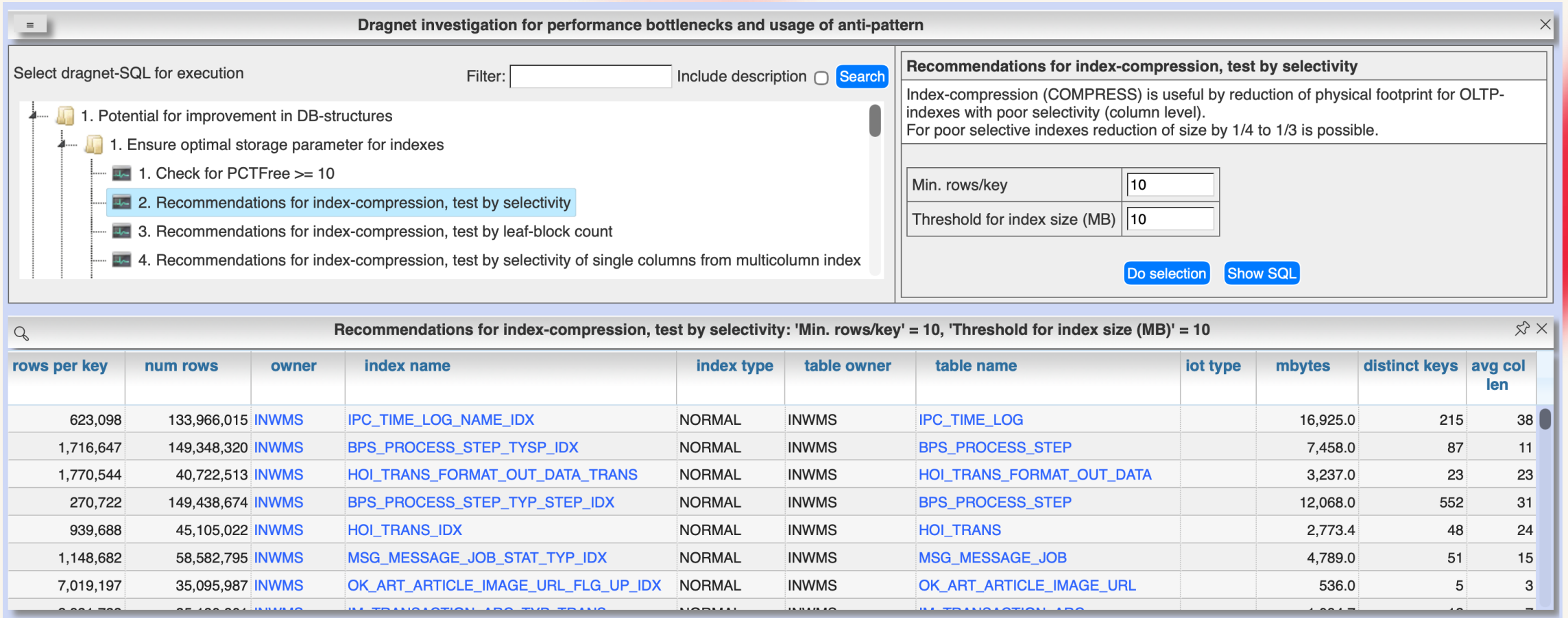
Based on active session history are shown outlier on database runtime per module je Module.
Units for time consideration are defined by date format picture of TRUNC-function (DD=day, HH24=hour etc.)

```
WITH Modules AS (
               SELECT /*+ PARALLEL(h,2) */
                      TRUNC(Sample_Time, picture)  Time_Range_Start,
                      Module,
                      MIN(Sample_Time)          First_Occurrence,
                      MAX(Sample_Time)          Last_Occurrence,
                      COUNT(*) * 10             Secs_Waiting
               FROM   DBA_Hist_Active_Sess_History h,
```

otto group one.O

1.1.2 .. 1.1.4 Recommendations for index compression

- The index key compression is available in all editions for decades now
- Nevertheless, it is often not used, but may save up to 30% or more of storage
- If not using the Advanced Compression Option, you need to specify the prefix length



rows per key	num rows	owner	index name	index type	table owner	table name	iot type	mbytes	distinct keys	avg col len
623,098	133,966,015	INWMS	IPC_TIME_LOG_NAME_IDX	NORMAL	INWMS	IPC_TIME_LOG		16,925.0	215	38
1,716,647	149,348,320	INWMS	BPS_PROCESS_STEP_TYSP_IDX	NORMAL	INWMS	BPS_PROCESS_STEP		7,458.0	87	11
1,770,544	40,722,513	INWMS	HOI_TRANS_FORMAT_OUT_DATA_TRANS	NORMAL	INWMS	HOI_TRANS_FORMAT_OUT_DATA		3,237.0	23	23
270,722	149,438,674	INWMS	BPS_PROCESS_STEP_TYP_STEP_IDX	NORMAL	INWMS	BPS_PROCESS_STEP		12,068.0	552	31
939,688	45,105,022	INWMS	HOI_TRANS_IDX	NORMAL	INWMS	HOI_TRANS		2,773.4	48	24
1,148,682	58,582,795	INWMS	MSG_MESSAGE_JOB_STAT_TYP_IDX	NORMAL	INWMS	MSG_MESSAGE_JOB		4,789.0	51	15
7,019,197	35,095,987	INWMS	OK_ART_ARTICLE_IMAGE_URL_FLG_UP_IDX	NORMAL	INWMS	OK_ART_ARTICLE_IMAGE_URL		536.0	5	3

1.2.3 **Detection of indexes with multiple indexed columns**

- Looks for indexes where one index indexes a subset of the columns of the other index, both starting with the same columns.

- The purpose of the index with the smaller column set can regularly be covered by the second index with the larger column set (including protection of foreign key constraints). So the first index often can be dropped without loss of function.

- The effect of less indexes to maintain and less objects in database cache with better cache hit rate for the remaining objects in cache is mostly higher rated than the possible overhead of using range scan on index with larger column set.

- A PK/ unique constraint on the smaller column set can be covered by the second index with the larger column set

1.2.4 **Detection of indexes by MONITORING USAGE**

- There are four reasons why an existing index should survive:
  - 1. It is used by SQL statements: then the index is not included in the list.
  - 2. Use for securing uniqueness by Unique Index, Unique or Primary Key Constraints
  - 3. Use for protection of a foreign key constraint
  - 4. Possible use for partition exchange if index structures are identical
- ALTER INDEX MONITORING USAGE; allows to exactly state used or not used
- The whole story about identifying unused indexes for DROP: https://rammpeter.blogspot.com/2019/12/oracle-db-identification-of-non.html

1.7.1 **Coverage of foreign-key relations by indexes (detection of potentially missing indexes)**

- Protection of columns with foreign key references by index can be necessary for:
  - Ensure delete performance of referenced table (suppress FullTable-Scan)
  - Suppress lock propagation (shared lock on index instead of table)

1.2.6 **Coverage of foreign-key relations by indexes (detection of potentially unnecessary indexes)**

- Protection of existing foreign key constraint by index on referencing column may be unnecessary if:
  - there are no physical deletes on referenced table
  - full table scan on referencing table is acceptable during delete on referenced table
  - possible shared lock issues on referencing table due to not existing index are no problem
- Especially for references from large tables to small master data tables often there's no use for the effort of indexing the referencing column.
- Due to the poor selectivity such indexes are mostly not useful for access optimization.

1.2.10 **Tables with PCT_FREE > 0 but without update-DML**

- For tables without updates you may consider setting PCTFREE=0
  and freeing this space by reorganizing this table.
- Free space in DB-blocks declared by PCT_FREE may be used for:
  - Reduce the risk of chained rows due to expansion of rowsize by update statements
  - Reduce the risk of ITL-waits by allowing the expansion of the ITL-list above INI_TRANS entries
- This selection shows candidates without any update statements since last analyze.
- If you don‘t need concurrent transactions in ITL-list above INI_TRANS
  then the recommendation is to set PCT_FREE=0

1.15 **Possibly expensive TABLE ACCESS BY INDEX ROWID with additional filter predicates on table**

- If in a SQL a table has additional filter conditions that are not covered by the used index you may consider extending the index by these filter conditions.

- This would ensure that you do the more expensive TABLE ACCESS BY ROWID only if that table row matches all your access conditions checked by the index.

2.1.3 **Full table scans with small cardinality: missing indexes?**

- Access by full table scan is often suboptimal if only a small part of a table is selected.
- They are out of place for OLTP-like access (small access time, many executions).
- Placing an index may reduce runtime significantly.
- Sorted by high runtime of full scan and small expected number of records.

2.4.2 Frequent access on small objects

- For frequently executed SELECT-statements on small objects it may be worth caching this content in the application instead of repeated access by SQL.
- This reduces CPU-contention and the risk of “Cache Buffers Chains” latch-waits.
- A remote application will benefit from suppressed network roundtrips too.
- Stored functions with function result caching or selects/subselects with result caching may also be used for this purpose

2.4.3 **Unnecessary high fetch count**

- For larger results per execution, it is worth accessing multiple records per fetch with bulk operation instead of single fetches.
- This results in only a slight reduction in CPU usage and SQL runtime.
- For remote clients it reduces the number of network roundtrips which may last longer than the fetch operation at DB itself.
- Specify the fetch size e.g. for:
  - SQL*Plus: > `SET ARRAYSIZE xy`
  - Java per statement: > `Statement.setFetchSize(xy);`
  - Java global:

```
Properties props = new Properties();
props.setProperty("user", "scott");
props.setProperty("password", "tiger");
props.setProperty("defaultRowPrefetch", "100"); // Oracle-specific
Connection conn = DriverManager.getConnection("jdbc:oracle:thin:…", props);
```

4.1.1 .. 4.1.5 Missing use of bind variables

- Using bind variables for volatile filter criteria in SQL seems to be standard, but it isn’t
- So, identifying the missing use of binds and pushing developers to fix this helps to:
  - Avoid SQL injection threat
  - Reduce hard parses needed for each execution with new values
  - Reduce memory requirements and pressure in SGA
- cursor_sharing=EXACT can reduce the problem, but with other side effects
- Detection of problematic missing use of bind variables can be done by evaluating:
  - the force_matching_signature which is equal for similar SQLs after literal replacement
  - the plan_hash_value to consider SQLs with same execution plan
  - similar substrings of SQL syntax (e.g. for PL/SQL without execution plan)

otto group one.O

4.2.2 **JDBC client statement cache probably not used**

- High soft parse rate on JDBC Thin connections suggests JDBC client-side statement caching is disabled or undersized.
- Enabling it keeps frequently used cursors permanently open on the connection.
- The JDBC statement cache is disabled per default in Oracle's JDBC driver.
  There are two ways to activate JDBC client-side statement caching and set cache size:

  1. Enable on JDBC connection:

```
((OracleConnection)conn).setImplicitCachingEnabled(true));
((OracleConnection)conn).setStatementCacheSize(100));
```

  2. Enable via JDBC URL (starting with DB release 19c):

```
jdbc:oracle:thin:@host:1521/srv?oracle.jdbc.implicitStatementCacheSize=100
```

otto group one.O

Thank you for your interest

Peter Ramm

Team Lead Architecture Consulting

Contact me via Teams

Mail: Peter.Ramm@og1o.de

BlueSky: https://bsky.app/profile/rammpeter.bsky.social

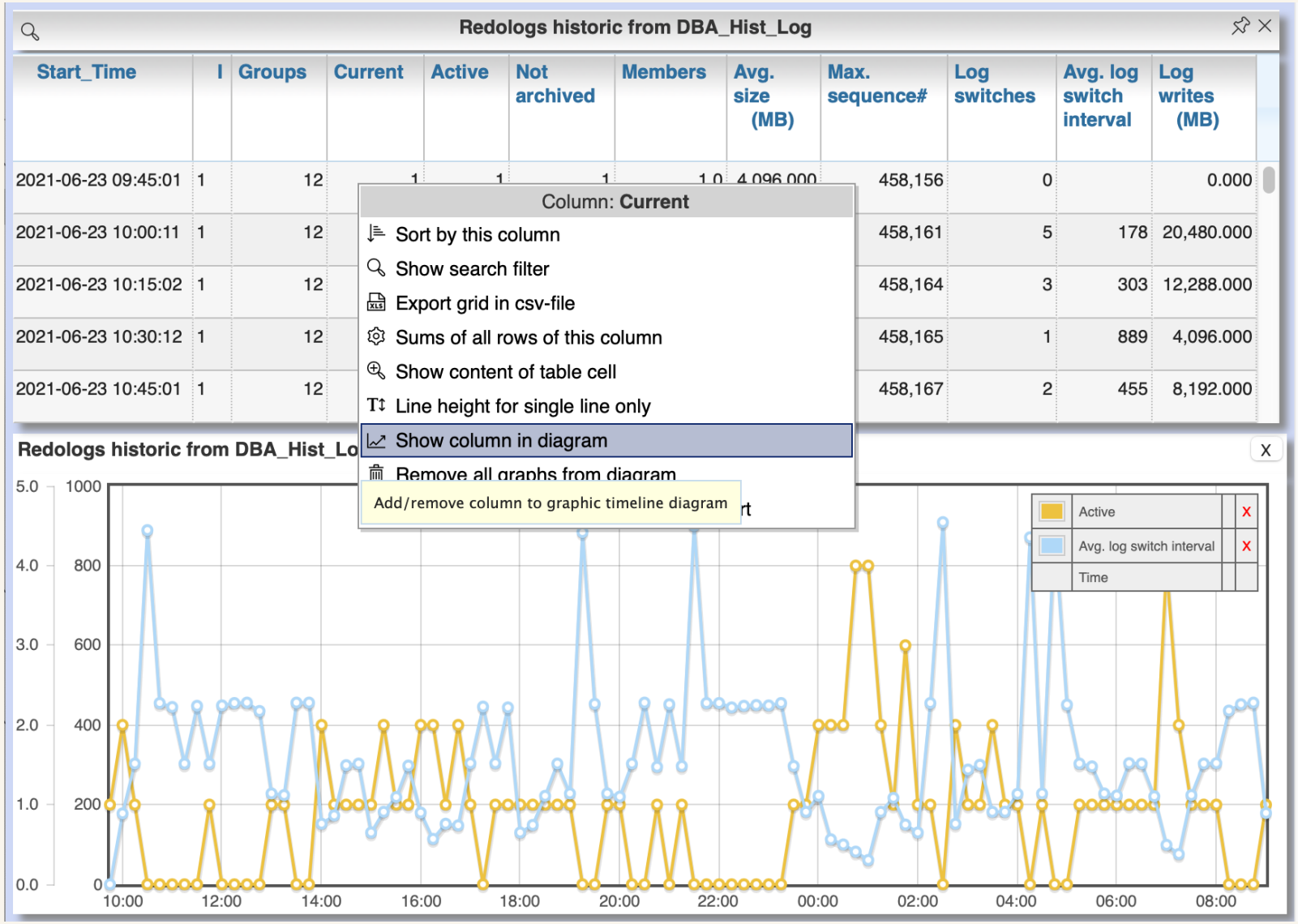
LinkedIn: https://www.linkedin.com/in/peter-ramm-ab842362/

otto group one.O

Backup

otto group one.O

Panorama for Oracle: presentation formats



Workflow in browser page continuously downwards

Results usually as tables

Time-related values can generally also be displayed as graphs by showing/hiding columns of the tables

Visualization in graphs via context menu (right mouse button)

Config setting PANORAMA_LOG_LEVEL=debug shows executed SQLs on console

otto group one.O